Name: Samir Kumar Gupta Roll No.: 2301730243 Environment: Ubuntu 24.04.5 LTS (OrbStack VM, arm64)

Question 4 - Docker network

Question 4: Docker Network - Demonstrate communication between two containers through a Docker network, using container names and container IP addresses.

Executed on an Ubuntu 24.04.5 LTS OrbStack machine (`ensp461-lab2`, arm64), Docker Engine 29.9.3. Source, logs and screenshots: https://github.com/ksamirdev/ensp461-lab2.

Setup

```
$ docker network create lab2-net
$ docker run -d --name n-a --network lab2-net ubuntu:24.04 sleep infinity
$ docker run -d --name n-b --network lab2-net ubuntu:24.04 sleep infinity
$ docker run -d --name n-c ubuntu:24.04 sleep infinity        # default bridge, for contrast
$ docker exec n-a sh -c 'apt-get update -qq && apt-get install -y -qq iputils-ping'
$ docker inspect n-a n-b --format '{{.Name}} {{(index .NetworkSettings.Networks "lab2-net").IPAddress}}'
/n-a 172.18.0.2
/n-b 172.18.0.3
```

Results

From	To	Result
n-a (lab2-net)	`n-b` by name	2/2 replies, resolved to 172.18.0.3
n-a (lab2-net)	`172.18.0.3` by IP	2/2 replies
n-c (default bridge)	`n-b` by name	`ping: n-b: Name or service not known`
n-c (default bridge)	`172.18.0.3` by IP	100% packet loss

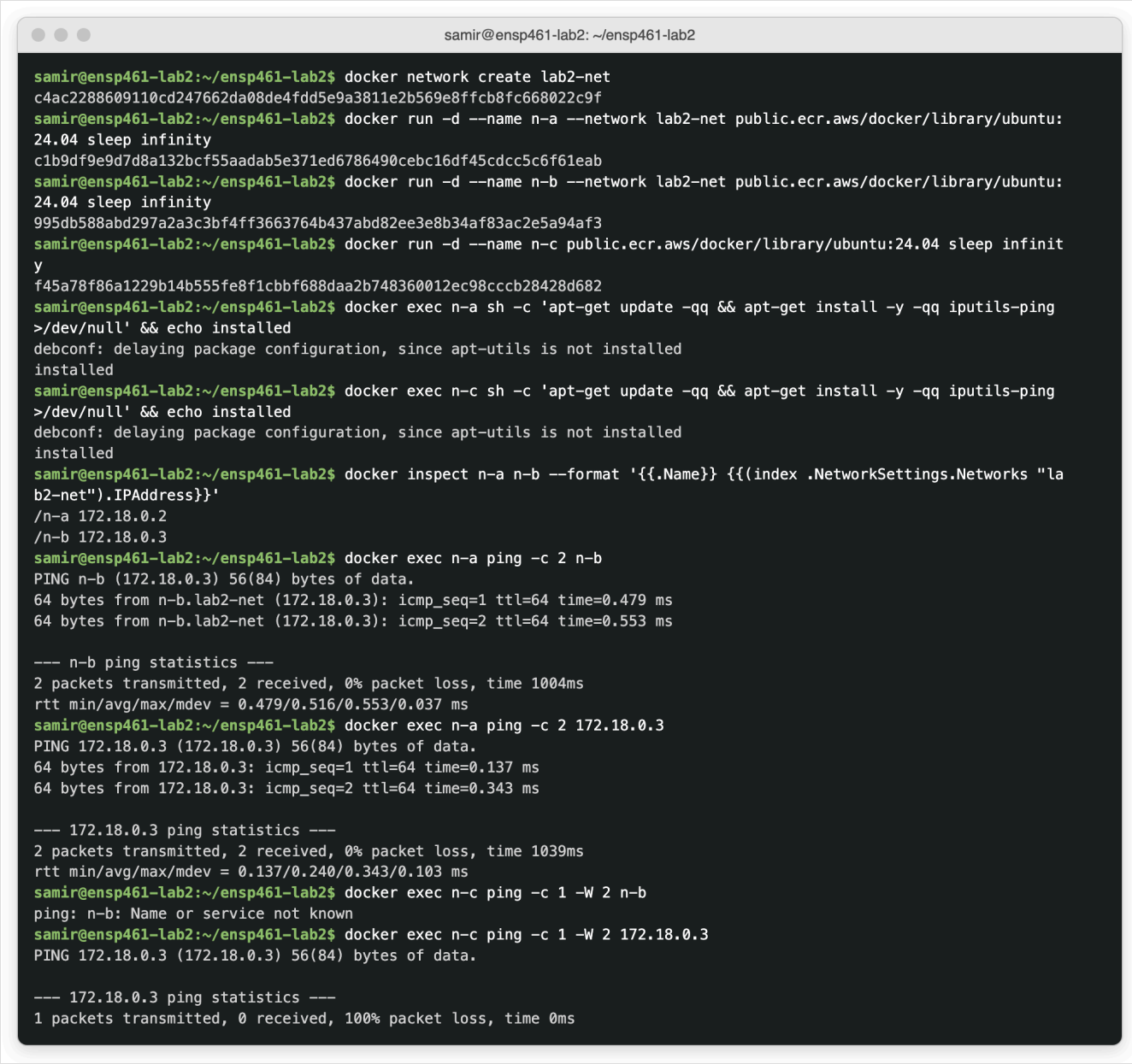


Fig: Network creation, IPs, ping by name and IP, isolation of the default bridge

Observations

- On a user-defined bridge network, Docker's embedded DNS server (`127.0.0.11`) resolves container names to their current IPs. Names are the reliable way to connect, because IPs are assigned at start and can change.
- The default `bridge` network has no name resolution between containers.
- Networks are isolated: `n-c` cannot reach `n-b` even by IP, because they sit on different bridges and Docker's iptables rules drop traffic between them.
- The same mechanism lets the Compose web service in Q2 reach Redis as `redis`, and lets the Minikube node reach the registry as `lab2-registry` in Q5.